SIMATS ENGINEERING

ASSESSMENT TOOL 1 **NAME : P SHASHANK**

COURSE NAME: Database Management Systems **REGNO:192524282**

COURSE CODE: CSA05

Q1

Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply

1.CREATE STUDENT TABLE

```
mysql> CREATE TABLE student (
    ->     RegNo INT PRIMARY KEY,
    ->     Name VARCHAR(20),
    ->     Gender CHAR(1),
    ->     DOB DATE,
    ->     MobileNo BIGINT,
    ->     City VARCHAR(20),
    ->     FacNo INT
    -> );
Query OK, 0 rows affected (0.13 sec)

mysql> DESC student;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| RegNo    | int         | NO   | PRI | NULL    |       |
| Name     | varchar(20) | YES  |     | NULL    |       |
| Gender   | char(1)     | YES  |     | NULL    |       |
| DOB      | date        | YES  |     | NULL    |       |
| MobileNo | bigint      | YES  |     | NULL    |       |
| City     | varchar(20) | YES  |     | NULL    |       |
| FacNo    | int         | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
7 rows in set (0.10 sec)
```

2CREATE DEPARTMENT TABLE

```
mysql> CREATE TABLE Department (
    ->     DeptNo INT PRIMARY KEY,
    ->     DeptName VARCHAR(20),
    ->     HOD VARCHAR(20)
    -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql> DESC Department;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| DeptNo   | varchar(4)  | NO   | PRI | NULL    |       |
| DeptName | varchar(15) | YES  |     | NULL    |       |
| DeptHead | varchar(4)  | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
3 rows in set (0.06 sec)
```

3. Insert Sample Data

Department

```
mysql> INSERT INTO Department VALUES
    -> (101,'CSE','Mohan'),
    -> (102,'ECE','Raju'),
    -> (103,'EEE','Ramesh');
Query OK, 3 rows affected (0.07 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM Department;
+--------+----------+--------+
| DeptNo | DeptName | HOD    |
+--------+----------+--------+
|    101 | CSE      | Mohan  |
|    102 | ECE      | Raju   |
|    103 | EEE      | Ramesh |
+--------+----------+--------+
3 rows in set (0.04 sec)
```

Student

```
mysql> INSERT INTO Student VALUES
    -> (19221151,'Akash','M','2004-01-12',9876543210,'Chennai',101),
    -> (19221152,'Priya','F','2005-02-13',9876543211,'Madurai',102),
    -> (19221153,'Rahul','M','2004-03-14',9876543212,'Salem',101),
    -> (19221154,'Sneha','F','2005-04-15',9876543213,'Coimbatore',103),
    -> (19221155,'Kiran','M','2004-05-16',9876543214,'Trichy',102);
Query OK, 5 rows affected (0.05 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> DESC STUDENT;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| RegNo    | int         | NO   | PRI | NULL    |       |
| Name     | varchar(20) | YES  |     | NULL    |       |
| Gender   | char(1)     | YES  |     | NULL    |       |
| DOB      | date        | YES  |     | NULL    |       |
| MobileNo | bigint      | YES  |     | NULL    |       |
| City     | varchar(20) | YES  |     | NULL    |       |
| FacNo    | int         | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
7 rows in set (0.01 sec)
```

JOIN Queries

1. Display Student Name and Department Name

```
mysql> SELECT Student.Name, Department.DeptName
    -> FROM Student
    -> INNER JOIN Department
    -> ON Student.DeptNo = Department.DeptNo;
+-------+----------+
| Name  | DeptName |
+-------+----------+
| Akash | CSE      |
| Priya | ECE      |
| Rahul | CSE      |
| Sneha | EEE      |
| Kiran | ECE      |
+-------+----------+
5 rows in set (0.00 sec)
```

Q2

Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.Bloom's Level: BL4 – Analyse

ANS:

Relations

STUDENT(RegNo, Name, Gender, DeptNo)

DEPARTMENT(DeptNo, DeptName, HOD)

Query 1

Retrieve the names of students who belong to the CSE department.

Relational Algebra:

πName(σDeptName='CSE'(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Query 2

Retrieve the names of female students in the CSE department.

Relational Algebra:

πName(σGender='F' ∧ DeptName='CSE'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Query 3

Retrieve the names of students whose HOD is 'Mohan'.

Relational Algebra:

πName(σHOD='Mohan'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Query 4

Retrieve the names and department names of all students.

Relational Algebra:

πName,DeptName

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT)

Query 5

Retrieve the department names having female students.

Relational Algebra:

πDeptName

(σGender='F'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Query 6

Retrieve the names of students who belong to the ECE department.

Relational Algebra:

πName

(σDeptName='ECE'

(STUDENT ⋈ STUDENT.DeptNo = DEPARTMENT.DeptNo DEPARTMENT))

Operators Used

Symbol	Meaning
σ	Selection (filters rows based on a condition)
π	Projection (selects required columns)
⋈	Join (combines two relations using a common attribute)
∧	Logical AND (multiple conditions)

Explanation

- **Selection (σ) is used to filter records based on one or more conditions.**
- **Projection (π) is used to retrieve only the required attributes.**
- **Join (⋈) combines the STUDENT and DEPARTMENT relations using the common attribute DeptNo.**

Bottom of Form

Q3

relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create

1. Create Student Table

```
mysql> CREATE TABLE student (
    ->     RegNo INT PRIMARY KEY,
    ->     Name VARCHAR(20),
    ->     Gender CHAR(1),
    ->     DOB DATE,
    ->     MobileNo BIGINT,
    ->     City VARCHAR(20),
    ->     FacNo INT
    -> );
Query OK, 0 rows affected (0.13 sec)

mysql> DESC student;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| RegNo    | int         | NO   | PRI | NULL    |       |
| Name     | varchar(20) | YES  |     | NULL    |       |
| Gender   | char(1)     | YES  |     | NULL    |       |
| DOB      | date        | YES  |     | NULL    |       |
| MobileNo | bigint      | YES  |     | NULL    |       |
| City     | varchar(20) | YES  |     | NULL    |       |
| FacNo    | int         | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
7 rows in set (0.10 sec)
```

2. Insert Sample Data

```
mysql> CREATE TABLE faculty
    -> (
    ->     RegNo INT(3),
    ->     Name VARCHAR(15),
    ->     Gender CHAR(1),
    ->     DOB DATE,
    ->     MobileNo BIGINT,
    ->     City VARCHAR(15)
    -> );
ERROR 1050 (42S01): Table 'faculty' already exists
mysql> INSERT INTO faculty
    -> VALUES
    -> (11911511,'mohan','m','2004-01-12',998826973,'mtr'),
    -> (11922112,'raju','m','2005-02-13',908658694,'mgr'),
    -> (11922113,'ramesh','m','2005-03-14',998263548,'mnr'),
    -> (11922154,'ramu','m','2006-04-14',908269279,'mgr'),
    -> (11922115,'san','m','2007-05-17',998269245,'hyt');
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> desc faculty;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| RegNo    | int         | YES  |     | NULL    |       |
| Name     | varchar(15) | YES  |     | NULL    |       |
| Gender   | char(1)     | YES  |     | NULL    |       |
| DOB      | date        | YES  |     | NULL    |       |
| MobileNo | bigint      | YES  |     | NULL    |       |
| City     | varchar(15) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

3. Display All Students

```
mysql> SELECT * FROM Student;
+----------+--------+--------+------------+-----------+------------+
| RegNo    | Name   | Gender | DOB        | MobileNo  | City       |
+----------+--------+--------+------------+-----------+------------+
| 19221151 | Mohan  | M      | 2004-01-12 | 990826973 | Madurai    |
| 19221152 | Raju   | M      | 2005-02-13 | 908658694 | Chennai    |
| 19221153 | Ramesh | M      | 2005-03-14 | 998263548 | Trichy     |
| 19221154 | Ram    | M      | 2006-04-14 | 908269279 | Salem      |
| 19221155 | San    | M      | 2007-05-17 | 998269245 | Coimbatore |
+----------+--------+--------+------------+-----------+------------+
5 rows in set (0.28 sec)
```

4. Display students from Chennai.

```
mysql> SELECT *
    -> FROM Student
    -> WHERE City='Chennai';
+----------+------+--------+------------+-----------+---------+
| RegNo    | Name | Gender | DOB        | MobileNo  | City    |
+----------+------+--------+------------+-----------+---------+
| 19221152 | Raju | M      | 2005-02-13 | 908658694 | Chennai |
+----------+------+--------+------------+-----------+---------+
1 row in set (0.06 sec)
```

Q4.Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution.Bloom's Level: BL4 – Analyze

Nested Subquery

```
mysql> SELECT * FROM Department;
+--------+----------+--------+
| DeptNo | DeptName | HOD    |
+--------+----------+--------+
|    101 | CSE      | Mohan  |
|    102 | ECE      | Raju   |
|    103 | EEE      | Ramesh |
+--------+----------+--------+
3 rows in set (0.00 sec)
```

Q5 Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency.Bloom's Level: BL5 – Evaluate

ANS;

```
mysql> SELECT s.Name, d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptNo = d.DeptNo;
+--------+----------+
| Name   | DeptName |
+--------+----------+
| Mohan  | CSE      |
| Raju   | ECE      |
| Ramesh | CSE      |
| Ram    | EEE      |
| San    | ECE      |
+--------+----------+
5 rows in set (0.05 sec)
```

Q6 .Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.Bloom's Level: BL6 – Create

ANS;

```
mysql> CREATE VIEW Student_View AS
    -> SELECT RegNo, Name, City
    -> FROM Student;
Query OK, 0 rows affected (0.12 sec)

mysql> SELECT * FROM Student_View;
+----------+--------+------------+
| RegNo    | Name   | City       |
+----------+--------+------------+
| 19221151 | Mohan  | Madurai    |
| 19221152 | Raju   | Chennai    |
| 19221153 | Ramesh | Trichy     |
| 19221154 | Ram    | Salem      |
| 19221155 | San    | Coimbatore |
+----------+--------+------------+
5 rows in set (0.03 sec)
```

Q7 Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation.Bloom's Level: BL4 – Analyze

ANS; Given Database Schema

Student

Student_ID	Student_Name	Dept_ID	Marks

Department

Dept_ID	Dept_Name

Example 1: Students in the CSE Department

Relational Algebra

$\pi Student\_Name, Dept\_Name(\sigma Dept\_Name='CSE'(Student \bowtie Department))$

Equivalent SQL

SQL

```
SELECT Student.Student_Name, Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID
WHERE Department.Dept_Name = 'CSE';
```

Analysis

- **Relational Algebra: Uses σ (Selection), ⋈ (Join), and π (Projection).**
- **SQL: Uses WHERE, JOIN, and SELECT.**

- Both return the same result, but SQL is more user-friendly and executable.

Example 2: Students Scoring More Than 80

Relational Algebra

$\pi_{Student\_Name, Marks}(\sigma_{Marks>80}(Student))$

Equivalent SQL

```
SELECT Student_Name, Marks
FROM Student
WHERE Marks > 80;
```

Analysis

- RA clearly separates selection and projection.

- SQL combines both operations into one query using SELECT and WHERE.

Example 3: Average Marks Department-wise

Relational Algebra (Extended)

$\gamma_{Dept\_ID, AVG(Marks) \rightarrow Avg\_Marks}(Student)$

Equivalent SQL :

```
SELECT Dept_ID,
AVG(Marks) AS Avg_Marks
FROM Student
GROUP BY Dept_ID;
```

Analysis

- **Extended RA uses the γ (Grouping) operator.**
- **SQL uses GROUP BY with aggregate functio**
- **SQL is easier to read and directly supported by DBMSs.**

Example 4: Student Names with Department Names

Relational Algebra

$\pi_{Student\_Name, Dept\_Name}(Student \bowtie Department)$

Equivalent SQL:

```
SELECT Student.Student_Name,
Department.Dept_Name
FROM Student
JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Analysis

- **RA uses the join operator ⋈.**

- SQL uses the JOIN ... ON clause.
- Both produce the same output.

Comparison: Relational Algebra vs SQL

Relational Algebra	SQL
Procedural query language	Declarative query language
Uses symbols like σ, π, ⋈	Uses keywords like SELECT, FROM, WHERE
Describes the sequence of operations	Describes the required result
Mainly used for query design and optimization	Used to interact with real databases
Theoretical foundation	Practical implementation

Database Schema

STUDENT(RegNo, Name, Gender, DOB, MobileNo, City)

Query 1

Display the names of students from Chennai.

Relational Algebra

$\pi_{Name}(\sigma_{City='Chennai'}(STUDENT))$

```
mysql> SELECT Name
    -> FROM Student
    -> WHERE City='Chennai';
+------+
| Name |
+------+
| Raju |
+------+
1 row in set (0.01 sec)
```

Query 2

Display the registration number and name of male students.

Relational Algebra

$\pi_{RegNo,Name}(\sigma_{Gender='M'}(STUDENT))$

```
mysql> SELECT RegNo, Name
    -> FROM Student
    -> WHERE Gender='M';
+----------+--------+
| RegNo    | Name   |
+----------+--------+
| 19221151 | Mohan  |
| 19221152 | Raju   |
| 19221153 | Ramesh |
| 19221154 | Ram    |
| 19221155 | San    |
+----------+--------+
5 rows in set (0.00 sec)
```

Query 3

Display all students born after 01-01-2005.

Relational Algebra

σDOB>'2005-01-01'(STUDENT)

```
mysql> SELECT *
    -> FROM Student
    -> WHERE DOB > '2005-01-01';
+----------+--------+--------+------------+-----------+------------+--------+
| RegNo    | Name   | Gender | DOB        | MobileNo  | City       | DeptNo |
+----------+--------+--------+------------+-----------+------------+--------+
| 19221152 | Raju   | M      | 2005-02-13 | 908658694 | Chennai    |    102 |
| 19221153 | Ramesh | M      | 2005-03-14 | 998263548 | Trichy     |    101 |
| 19221154 | Ram    | M      | 2006-04-14 | 908269279 | Salem      |    103 |
| 19221155 | San    | M      | 2007-05-17 | 998269245 | Coimbatore |    102 |
+----------+--------+--------+------------+-----------+------------+--------+
4 rows in set (0.06 sec)
```

Q8 .Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements.

Bloom's Level: BL5 – Evaluate

ANS: Query 1: Inefficient Query (Using Subquery)

Problem: Display the names of students who belong to the CSE department.

```
mysql> SELECT Name
    -> FROM Student
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM Department
    -> WHERE DeptName = 'CSE'
    -> );
+--------+
| Name   |
+--------+
| Mohan  |
| Ramesh |
+--------+
2 rows in set (0.06 sec)
```

Query 2: Optimization Using View

Create a view to simplify repeated queries.

```
mysql> CREATE VIEW Student_Department AS
    -> SELECT s.RegNo,
    ->        s.Name,
    ->        d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptNo = d.DeptNo;
Query OK, 0 rows affected (0.03 sec)

mysql> SELECT * FROM Student_Department;
+----------+--------+----------+
| RegNo    | Name   | DeptName |
+----------+--------+----------+
| 19221151 | Mohan  | CSE      |
| 19221153 | Ramesh | CSE      |
| 19221152 | Raju   | ECE      |
| 19221155 | San    | ECE      |
| 19221154 | Ram    | EEE      |
+----------+--------+----------+
5 rows in set (0.06 sec)
```

Q9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem

Scenario

A Student Management System stores student records. The administrator wants to know the number of students in each city and identify cities that have more than one student.

Query 1: Grouping and Aggregation

Display the number of students in each city.

```
mysql> SELECT City, COUNT(*) AS Total_Students
    -> FROM Student
    -> GROUP BY City;
+------------+----------------+
| City       | Total_Students |
+------------+----------------+
| Madurai    |              1 |
| Chennai    |              1 |
| Trichy     |              1 |
| Salem      |              1 |
| Coimbatore |              1 |
+------------+----------------+
5 rows in set (0.06 sec)
```

Q10. Given multiple relations, analyze the query requirements and decide whether to use Views, JOINs, or Subqueries, justifying your choice with reasoning.

Bloom's Level: BL5 – Evaluate

Relations

STUDENT(RegNo, Name, Gender, DeptNo, City)

DEPARTMENT(DeptNo, DeptName, HOD)

1. Using JOIN

Query

Display the student name along with the department name.

```
mysql> SELECT s.RegNo, s.Name, d.DeptName
    -> FROM Student s
    -> INNER JOIN Department d
    -> ON s.DeptNo = d.DeptNo;
+----------+--------+----------+
| RegNo    | Name   | DeptName |
+----------+--------+----------+
| 19221151 | Mohan  | CSE      |
| 19221153 | Ramesh | CSE      |
| 19221152 | Raju   | ECE      |
| 19221155 | San    | ECE      |
| 19221154 | Ram    | EEE      |
+----------+--------+----------+
5 rows in set (0.00 sec)
```

Reason

- **JOIN is used because data is required from both Student and Department tables.**
- **It is faster and more efficient for retrieving related data from multiple tables.**

- ## 2. Using SUBQUERY
- **Query**
- **Display the names of students who belong to the CSE department.**

```
mysql> SELECT Name
    -> FROM Student
    -> WHERE DeptNo =
    -> (
    -> SELECT DeptNo
    -> FROM Department
    -> WHERE DeptName = 'CSE'
    -> );
+--------+
| Name   |
+--------+
| Mohan  |
| Ramesh |
+--------+
2 rows in set (0.05 sec)
```